

PH Stocks Advisor — High-Level Design (HLD)

System	PH Stocks Advisor — agentic AI advisor for PSE-listed stocks
Document	High-Level Design
Status	Current as of 2026-07-23
Repository	<code>agentic-ph-stocks-advisor</code>
Production	<code>https://phstockadvisor.sakayandgo.com</code> (Docker Compose behind a Cloudflare tunnel)

1. Purpose and scope

1.1 Purpose

Provide retail investors with a plain-English, multi-dimensional analysis of Philippine Stock Exchange (PSE) stocks, concluding in a **0–100 verdict score** on a buy-decision scale (**AVOID · DON'T BUY · WAIT · BUY · STRONG BUY**), with live progress streaming, per-trading-day caching, user accounts, and a private portfolio advisory for elevated users.

1.2 In scope

Web application, background analysis pipeline, market-data acquisition layer (MCP), LLM agent framework, persistence, authentication, admin back-office, deployment and CI/CD.

1.3 Out of scope

Order execution/brokerage integration, real-time tick data, mobile native apps, financial advice compliance workflows (the product displays an informational disclaimer).

2. System overview

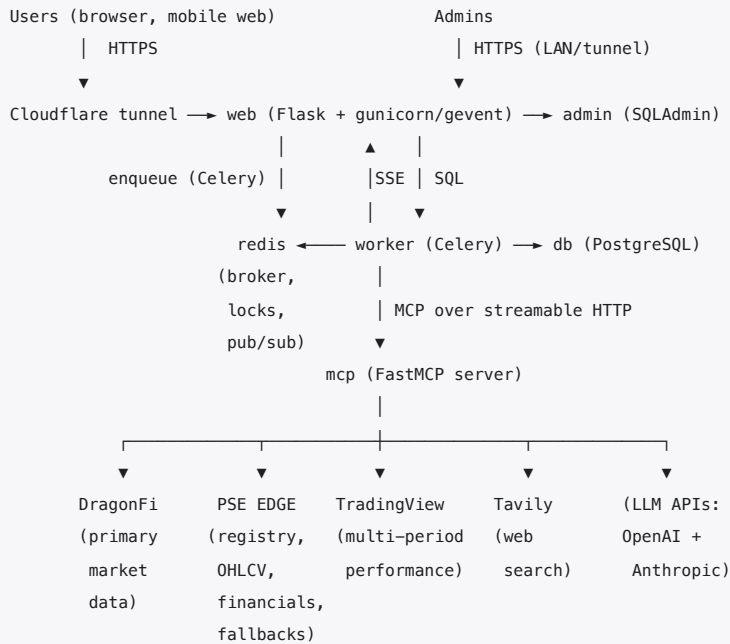
A user submits a ticker. The system validates it, fans out **six specialist LLM agents in parallel** — price, dividends, price movement, valuation, controversy/risk, macro sentiment — each grounded in deterministic data payloads fetched from market-data sources through an **MCP (Model Context Protocol) server**. A **consolidator agent** synthesises the analyses into a single report; a **deterministic weighted average** of six LLM-judged sub-scores produces the verdict score. Results stream to the browser over SSE and are cached per trading day.

Core design principles

Principle	Consequence
LLMs judge, code computes	All load-bearing arithmetic (ratios, weighted score, verdict derivation) is deterministic Python
Agent data contract	Prompts reference only fields present in the serialised payload; facts (e.g. REIT status) are data, never inference
Honest degradation	Missing data → disclosed gaps excluded from the score; upstream outages → transient, retryable errors, never false negatives
Single choke points	Rendering invariants live in <code>parse_sections</code> ; model selection in <code>build_chat_model</code> ; freshness in <code>trading_calendar</code>

3. Architecture

3.1 Context / container view



Seven Compose services: `db`, `redis`, `mcp`, `web`, `worker`, `admin`, `advisor` (one-shot CLI). Production runs the same topology from pre-built GHCR images (`docker-compose.prod.yml`) behind `cloudflared` ; the tunnel is the only ingress.

3.2 Component view

Component	Technology	Responsibilities
Web app	Flask 3, gunicorn + gevent workers	Routing, auth (passkeys + OAuth), trading-session gating, rate limiting, same-ticker dedup, SSE relay, report rendering, holdings/portfolio API
Analysis worker	Celery 5 (Redis broker)	Executes <code>analyse_stock</code> / <code>portfolio_analyse_stock</code> ; owns the LangGraph run; publishes progress; persists results
Agent framework	LangGraph + LangChain	<code>validate</code> → 6-way parallel fan-out → <code>consolidate</code> graph; typed state with reducer channels; graceful-degradation fallbacks
LLM layer	<code>langchain-openai</code> + <code>langchain-anthropic</code>	Provider-agnostic factory <code>build_chat_model([provider:]tier)</code> ; three tiers per provider; per-agent assignment (<code>AGENT_LLM_SPECS</code>) allowing mixed providers in one run
MCP server	FastMCP (streamable HTTP)	Sole data path; exposes <code>validate_symbol</code> + six <code>fetch_*</code> tools; typed error marker protocol for not-found
Data layer	requests + scrapers	Services (one per dimension) orchestrating clients: DragonFi (primary), PSE EDGE x3 (registry/OHLCV/dividends/financials — also outage fallback), TradingView, Tavily
Persistence	PostgreSQL (prod) / SQLite (dev, CLI)	Repository ABC + two implementations; idempotent migrations at startup
Admin	SQLAdmin (separate container)	Users (type promotion), passkey credential revocation, report inspection
Frontend	Jinja2 + vanilla JS	Tala design system (token-driven CSS); SSE client with polling fallback; WebAuthn ceremonies; report visual decoration
Observability	Langfuse (optional), structured logging, <code>/healthz</code>	LLM tracing; Docker/uptime probes

3.3 Key architectural decisions (ADR summary)

Decision	Rationale
MCP as the only data path	One network boundary to mock/trace; services reusable by CLI, web, and any future MCP consumer
Verdict = deterministic weighted score, binary derived	Meter/labels/chips can never contradict; tuning via env weights + prompt rubrics, no retraining
Same-ticker dedup via Redis <code>SET NX</code> + pre-generated task id	Identical concurrent requests cost one LLM run; joiners stream the winner's progress
Trading-calendar freshness (3 PM PHT close, weekend roll-back)	Nothing changes between close and next open; Friday's report serves the whole weekend
Graceful degradation with <code>data_gaps</code> channel	A dividend-less stock or one broken source yields an honest partial report, not a failure
PSE EDGE fallbacks (validation, price, financials, REIT registry)	Survives total outage of the primary vendor, proven live during the 2026-07-23 DragonFi HTTP-515 incident
Passkeys primary, OAuth as recovery	Passwordless with no account prerequisite; lost device ≠ lockout
Content-hash static asset versioning	Cache-busting independent of release cadence (app version only bumps on GitHub Releases)

4. Data design

4.1 Storage schema (both backends, idempotent migrations)

Table	Purpose	Notable columns
<code>reports</code>	One row per completed analysis (shared across users)	<code>symbol</code> , <code>verdict</code> (binary, legacy-compatible), <code>score</code> (0–100, nullable for pre-scoring rows), <code>summary</code> , six <code>*_section</code> texts, <code>created_at</code>
<code>users</code>	Account registry (passkey + OAuth)	<code>oid</code> (PK; <code>passkey:<uuid></code> or provider subject), <code>email</code> , <code>provider</code> , <code>user_type</code> (0 normal / 1 elevated), login timestamps
<code>webauthn_credentials</code>	Passkey public keys	<code>credential_id</code> , <code>user_oid</code> , <code>public_key</code> (public material only), <code>sign_count</code> , <code>transports</code> , <code>aaguid</code>
<code>user_symbols</code>	Per-user analysed-ticker history (drives sidebar)	<code>user_id</code> (email), <code>symbol</code> , timestamps
<code>holdings</code>	Elevated users' positions	<code>user_id</code> , <code>symbol</code> , <code>shares</code> , <code>avg_cost</code>
<code>portfolio_reports</code>	Private portfolio advisories	<code>user_id</code> , <code>symbol</code> , <code>analysis</code> , <code>base_report_id</code>

4.2 Redis keyspace

Key pattern	Purpose	Lifetime
<code>analysis:inflight:<SYMBOL></code>	Same-ticker dedup lock (value = task id)	10 min TTL + eager clear
<code>analysis:task:<task_id></code>	Reverse mapping for cancellation	10 min TTL
<code>analysis:progress:<task_id></code> (+ pub/sub channel)	SSE state snapshot + live events	Run duration
<code>ratelimit:<user>:<cutoff-day></code>	Daily analysis counter (atomic Lua reserve/release)	Until next 3 PM PHT cutoff
<code>portfolio:inflight:<user>:<symbol></code>	Portfolio-run tracking for page refresh	10 min TTL

4.3 Domain model (Pydantic)

Payload models per dimension (`StockPrice` , `DividendInfo` — incl. NI-based `payout_ratio` and cash-based `fcf_payout_ratio` — , `PriceMovement` , `FairValueEstimate` (+ `is_reit`), `ControversyInfo` , `SentimentInfo` (+ `bsp_rate` , `is_reit`)); agent wrappers (`*Analysis` = `data` + `prose`); `AdvisorState` (consolidator input incl. `data_gaps`); `ConsolidationResponse` (structured LLM output: verdict claim, summary, six optional sub-scores); `FinalReport` ; and `SCORE_BANDS` / `score_band()` mapping scores to display bands.

5. Interface design

5.1 Public HTTP interface (selected)

Endpoint	Method	Purpose
<code>/analyse</code>	POST	Start/join/cache-hit an analysis (dedup + rate limit + session gate)
<code>/stream/<task_id></code>	GET (SSE)	Live progress: snapshot replay + step events + terminal event
<code>/status/<task_id></code>	GET	Polling fallback; terminal payload includes <code>verdict</code> , <code>score</code> , <code>report_id</code>
<code>/report/<symbol></code>	GET	Rendered report (verdict panel, meter, agent cards)
<code>/history/<symbol></code>	GET	Prior reports with band chips
<code>/auth/passkey/*</code>	POST	WebAuthn register/login (begin/complete), list/delete
<code>/auth/*</code>	GET/POST	OAuth recovery (Entra ID, Google)
<code>/api/holdings/<symbol></code>	GET/POST/DELETE	Elevated: position CRUD
<code>/api/portfolio-analyse/<symbol></code> , <code>/api/portfolio-report/<symbol></code>	POST/GET	Elevated: advisory run + fetch
<code>/healthz</code>	GET	Public liveness (DB + Redis checks)

5.2 MCP tool interface

`validate_symbol` , `fetch_stock_price` , `fetch_dividend_info` , `fetch_price_movement` , `fetch_fair_value` , `fetch_controversy_info` , `fetch_sentiment_info` — all keyed by `symbol` ; typed payloads returned as structured content; not-found signalled by a `SymbolNotFoundError`: message marker that the client re-raises as a typed exception (marker matched anywhere in the message — the MCP framework wraps tool errors).

5.3 External interfaces

Provider	Interface	Role
DragonFi	REST (<code>api.dragonfi.ph/api/v2</code>)	Primary: profiles, listing universe, valuation multiples, financial trends
PSE EDGE	AJAX endpoints + HTML scraping	Authoritative registry (autocomplete), OHLCV, dividend declarations, stockData snapshot, audited annual financials, REIT identification — validation/price/financials fallback
TradingView	Scanner API	Multi-period performance/volatility
Tavily	Search API	News enrichment, global events, BSP policy-rate context
OpenAI / Anthropic	LangChain chat APIs	Per-agent LLMs (three tiers per provider)

6. Technology stack

Python ≥ 3.14 · Flask 3 · Celery 5 + Redis 6.4 (kombu pins redis < 6.5) · PostgreSQL / SQLite · LangGraph 1.x + LangChain 1.x · `langchain-openai` / `langchain-anthropic` · FastMCP (`mcp` ≥ 1.28) · webauthn 3 + MSAL + Google OAuth · gunicorn + gevent · Jinja2 + vanilla JS (no frontend framework) · fpdf2 (PDF export) · Langfuse (optional tracing) · pytest / ruff / pyright · uv (dependency compilation) · Docker Compose · GitHub Actions + GHCR · Cloudflare tunnel.

7. Security design

- **Authentication:** WebAuthn passkeys primary (open self-signup, email-first); server verifies against the **configured** RP ID/origin, never request headers (safe behind the tunnel). OAuth (Entra ID, Google) retained as recovery. Session cookies via Flask-Session (Redis-backed in prod).
- **Anti-enumeration:** unknown emails receive deterministic decoy credential lists; all registration/login failures share uniform generic messages.
- **Account-takeover guard:** anonymous callers can never attach a passkey to an existing email; adding devices requires an authenticated session.
- **Authorisation:** `user_type` gates elevated features (unlimited runs, cache bypass, portfolio advisory); admins promote via the back-office; login upserts never overwrite `user_type`.
- **CSRF:** token in session + `X-CSRFToken` header on state-changing JS calls.
- **Secrets:** environment-injected at runtime; never baked into images; compose forwards an explicit allowlist; the app fails fast when the *selected* LLM provider's key is absent.
- **Data stored:** passkey **public** keys only; no passwords anywhere; reports are shared but per-user visibility is scoped by `user_symbols`; portfolio data is private per user.

8. Performance and scalability

- **Caching:** per-trading-day report cache (freshness = last 3 PM PHT close, weekend roll-back); no analyses during market hours; process-lifetime caches for stable lookups (listing universe — non-empty only, EDGE cmpy-ids, REIT registry answers).
- **Cost control:** same-ticker dedup collapses concurrent demand to one LLM run; per-user daily rate limits (atomic Lua reserve); per-agent LLM tiers put cheap models on specialists and strong models on consolidation.
- **Concurrency:** gevent web workers stream many SSE connections cheaply; Celery parallelism scales analysis throughput horizontally (`docker compose up -d --scale worker=N`); the six specialists execute as a parallel LangGraph fan-out within a run.
- **Frontend:** content-hashed static assets (immutable-cache friendly, CDN-safe).

9. Reliability and resilience

- **Vendor outage:** validated live — with DragonFi fully down, ticker validation, price, valuation (audited EPS/BVPS Graham estimate), income trends, declared dividends, and REIT status all continue via PSE EDGE; remaining gaps are disclosed, scores renormalise.
- **Failure semantics:** transient vs definitive errors are distinct types end-to-end; empty upstream caches are never pinned; the systemic-failure guard aborts rather than fabricates when *all* dimensions fail; all dedup/rate-limit claims are released on every failure path; inflight locks carry TTLs.
- **Idempotent operations:** schema migrations, lock cleanup, and SSE snapshot replay all tolerate retries and restarts.

10. Observability

`/healthz` (DB + Redis) drives container healthchecks and uptime probes; structured logs per service; optional Langfuse tracing wraps every LLM call with per-run metadata; Celery task-level logging records verdict/score/report id per analysis; an optional OpenTelemetry overlay (collector + Jaeger + Prometheus + Grafana) exists for deep dives.

11. Deployment and CI/CD

- **Environments:** local dev (Compose, hot SQLite optional), production (Compose from GHCR images + cloudflared; only the tunnel is exposed).
- **Pipeline:** `develop` → CI (ruff, pyright, unit + integration tests) → merge-commit-only promotion to `main` → path-gated multi-arch image builds (app image rebuilds only when its build inputs change; admin separately) → GHCR push → optional Azure/SSH deploys.
- **Versioning:** fully automatic and **tag-driven** — every merge to `main` that rebuilds the app image mints an annotated `vX.Y.Z` tag (patch+1, floored by the committed `pyproject.toml` version), bakes it into the build, tags the image `latest + X.Y.Z`, and verifies the published artifact reports that version. App version, git tag, and image tag cannot drift; `GET /version` reports the running build. Static-asset cache keys remain content hashes, independent of version.
- **Dependency locking:** `uv pip compile --extra=postgres` produces `requirements.txt` (the `postgres` extra carries `psycopg2` and is mandatory).

12. Constraints, assumptions, risks

Item	Type	Notes
Redis ceiling 6.4.x	Constraint	<code>celery[redis]</code> → kombu pins <code>redis < 6.5</code> ; upgrading to Redis 8 requires replacing the Celery broker
PSE EDGE scraping	Risk	HTML/label changes break fallback parsers; mitigated by fixture-pinned tests and honest-gap degradation
EDGE autocomplete omits preferred shares	Constraint	An EDGE no-match is never treated as definitive; definitive rejection requires the DragonFi universe
DragonFi single primary vendor	Risk (mitigated)	EDGE fallbacks cover validation/price/financials/REIT; dividend yield/payout still DragonFi-only
LLM output variability	Risk	Structured output + deterministic scoring + render-time section filtering bound the blast radius
PSE holiday calendar	Assumption	Freshness handles weekends; exchange holidays are a manual <code>NON_TRADING_DATES</code> list (currently empty)
Single-region deployment	Constraint	One production host behind a tunnel; DB backups/DR are operational concerns outside this design

13. Future considerations (recorded backlog)

True FFO via depreciation data (real 17-Q parsing), an authoritative BSP/PDS rates client (replacing web-search rate context and enabling a DDM discount rate), property-level REIT fundamentals (occupancy/WALE/tenant mix), account linking across auth providers (email-collision edge), and PSE holiday calendar population.